TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

NGÔN NGỮ LẬP TRÌNH

C++

TỪ CON SỐ 0 ĐẾN THI ĐẤU

Dành cho người chưa từng viết một dòng code nào

Mọi đoạn code đều đã biên dịch và chạy thật với g++ (C++17)

NỘI DUNG

Chương trình đầu tiên · Biến · Kiểu dữ liệu · Tràn số
Nhập xuất · Toán tử · Rẽ nhánh · Vòng lặp
Mảng · vector · Hàm · Tham chiếu · Đệ quy
Xâu kí tự · struct · sort · pair
Kĩ thuật thi đấu · Tìm & sửa lỗi (stress test, sanitizer)
25 cái bẫy · Sổ tay cú pháp · Lộ trình 30 ngày

Biên soạn: Lê Thành Nam

Lê Thành Nam

MỤC LỤC

GIỚI THIỆU 7
Tài liệu này dành cho ai? 7
Tài liệu này KHÔNG dạy gì - và vì sao 7
Vì sao là C++ mà không phải ngôn ngữ khác? 7
Bạn cần chuẩn bị gì 8
Cách đọc tài liệu này 8
Phần 1. Chương trình đầu tiên 9
1.1. Chương trình máy tính là gì? 9
1.2. Chương trình đầu tiên 9
1.3. Giải thích từng dòng 9
Dòng 1: #include <iostream> 9
Dòng 2: using namespace std; 9
Dòng 4: int main() { 10
Dòng 5: cout << "Xin chao the gioi"; 10
Dòng 6: return 0; 10
1.4. Dấu chấm phẩy - kẻ thù số một của người mới 10
1.5. Chú thích - viết cho người, máy bỏ qua 11
1.6. Biên dịch - chuyện gì xảy ra khi bạn bấm Run 11
1.7. Ba loại lỗi - phân biệt cho đúng 11
Loại 1: Lỗi biên dịch (compile error) 12
Loại 2: Lỗi khi chạy (runtime error) 12
Loại 3: Lỗi logic - kẻ thù thật sự 12
1.8. Bài tập 12
Phần 2. Biến và kiểu dữ liệu 13
2.1. Biến là gì? 13
Vì sao phải khai báo kiểu? 13
2.2. Quy tắc đặt tên biến 13
2.3. int - số nguyên, kiểu dùng nhiều nhất 14
2.4. long long - khi 2 tỉ là không đủ 14
2.5. Tràn số - cái bẫy giết chết nhiều bài thi nhất 14
Bẫy tinh vi hơn: tính bằng int rồi mới gán vào long long 14
2.6. double - số thực 15
2.7. Số thực KHÔNG chính xác - sự thật gây sốc 15
Hệ quả: không bao giờ so sánh double bằng == 15
2.8. char - một kí tự, và nó thật ra là một con số 16
2.9. bool - đúng hoặc sai 16
2.10. Ép kiểu - chuyển từ kiểu này sang kiểu kia 17

Lê Thành Nam

2.11. Chia số nguyên - cái bẫy kinh điển ... 17
Phép chia lấy dư % và chuyện số âm ... 17
2.12. Hằng số - thứ không được đổi ... 18
2.13. Bảng tổng kết kiểu dữ liệu ... 18
2.14. Bẫy của Phần 2 ... 18
2.15. Bài tập ... 18
Phần 3. Nhập và xuất ... 20
3.1. cin - đọc từ bàn phím ... 20
3.2. cin bỏ qua mọi khoảng trắng ... 20
3.3. Đọc cho tới khi hết dữ liệu ... 20
3.4. Đọc xâu - hai cách khác nhau hoàn toàn ... 21
cin >> chỉ đọc MỘT TỪ ... 21
getline đọc CẢ DÒNG ... 21
3.5. Cái bẫy getline - kinh điển đến mức ai cũng dính ... 21
3.6. Định dạng số thực khi in ... 22
3.7. Vài cách in khác ... 22
endl hay \n? ... 23
3.8. Tăng tốc nhập xuất - hai dòng cứu mạng ... 23
3.9. Đọc và ghi file ... 23
3.10. Khung chương trình chuẩn - chép vào sổ tay ... 24
3.11. Bẫy của Phần 3 ... 24
3.12. Bài tập ... 25
Phần 4. Toán tử ... 26
4.1. Toán tử số học ... 26
4.2. Gán và gán rút gọn ... 26
4.3. Tăng và giảm: ++i và i++ ... 26
4.4. Toán tử so sánh ... 27
4.5. Bẫy chết người: = và == ... 27
4.6. Bẫy: so sánh dây chuyền ... 27
4.7. Toán tử logic ... 28
Ngắt ngắn mạch - tính chất cực kì hữu ích ... 28
4.8. Thứ tự ưu tiên ... 28
4.9. Toán tử ba ngôi ... 29
4.10. Toán tử bit ... 29
Phần 5. Rẽ nhánh ... 31
5.1. if - nếu ... 31
5.2. if / else - nếu... không thì ... 31
5.3. else if - nhiều nhánh ... 31
5.4. Bẫy: dấu chấm phẩy sau if ... 31
5.5. Bẫy: quên ngoặc nhọn ... 32

Lê Thành Nam

5.6. switch - chọn theo giá trị ... 32
Bẫy: quên break ... 32
5.7. Kết hợp điều kiện ... 33
5.8. Bẫy của Phần 4 và 5 ... 33
5.9. Bài tập ... 33
Phần 6. Vòng lặp ... 34
6.1. Vì sao cần vòng lặp ... 34
6.2. for - vòng lặp dùng nhiều nhất ... 34
Các dạng for thường gặp ... 34
6.3. Bẫy: dấu chấm phẩy sau for ... 35
6.4. while - lặp khi chưa biết trước số lần ... 35
6.5. do-while - chạy trước, hỏi sau ... 35
6.6. break và continue ... 36
6.7. Bẫy: break chỉ thoát MỘT vòng ... 36
6.8. Vòng lặp lồng nhau ... 36
6.9. Bẫy: biến đếm kiểu double ... 37
6.10. for với nhiều biến ... 37
6.11. Những mẫu vòng lặp phải thuộc ... 37
Tính tổng ... 37
Đếm theo điều kiện ... 37
Tìm giá trị lớn nhất ... 37
Đếm chữ số / tách chữ số ... 38
Kiểm tra số nguyên tố ... 38
6.12. Đo độ phức tạp - kĩ năng sống còn ... 38
6.13. Bẫy của Phần 6 ... 39
6.14. Bài tập ... 39
Phần 7. Mảng ... 40
7.1. Vì sao cần mảng ... 40
7.2. Khai báo và khởi tạo ... 40
7.3. Chỉ số bắt đầu từ 0 - nguồn cơn của mọi lỗi ... 40
Ra ngoài mảng - chuyện gì xảy ra? ... 41
7.4. Mảng toàn cục tự động bằng 0 - mảng cục bộ thì không ... 41
7.5. Mảng lớn phải để toàn cục - nếu không sẽ SẬP ... 41
7.6. Duyệt mảng ... 42
Bẫy: range-for chép giá trị ... 42
7.7. Mảng hai chiều ... 42
7.8. memset - và cái bẫy của nó ... 43
7.9. sizeof - đo kích thước ... 43
7.10. vector - mảng biết co giãn ... 43
Bẫy: v.size() là số KHÔNG DẤU ... 44

Lê Thành Nam

vector hai chiều 44
7.11. Mảng thường hay vector? 45
7.12. Vài việc hay làm với mảng 45
7.13. Bẫy của Phần 7 46
7.14. Bài tập 46
Phần 8. Hàm và đệ quy 47
8.1. Hàm là gì? 47
8.2. Vì sao nên viết hàm? 47
8.3. Hàm không trả về gì: void 47
8.4. Truyền giá trị và truyền tham chiếu - mục quan trọng nhất 48
Vì sao điều này quan trọng cho tốc độ 48
8.5. Nạp chồng hàm 48
8.6. Tham số mặc định 49
8.7. Phạm vi biến 49
8.8. Đệ quy - hàm tự gọi chính nó 49
8.9. Đệ quy ngây thơ chậm tới mức nào 50
Cách sửa: ghi nhớ kết quả 51
8.10. Đệ quy hay vòng lặp? 51
8.11. Bẫy của Phần 8 51
8.12. Bài tập 52
Phần 9. Xâu kí tự 53
9.1. Tạo xâu 53
9.2. Nối xâu - dễ đến bất ngờ 53
9.3. Độ dài và truy cập 53
9.4. So sánh xâu 54
9.5. Tìm kiếm và npos 54
9.6. Cắt xâu con 55
9.7. Đổi giữa số và xâu 55
9.8. Kiểm tra loại kí tự 55
9.9. Duyệt xâu 56
9.10. Những việc hay làm với xâu 56
Đếm số lần xuất hiện của mỗi chữ cái 56
Kiểm tra xâu đối xứng 56
Sắp xếp 57
Tách từ bằng stringstream 57
9.11. Bảng tra cứu string 57
9.12. Bẫy của Phần 9 58
9.13. Bài tập 58
Phần 10. struct và bước đầu vào STL 59
10.1. Vấn đề: dữ liệu đi thành bộ 59

Lê Thành Nam

10.2. struct - tự tạo kiểu dữ liệu 59
10.3. sort - hàm quý nhất trong C++ 60
10.4. Sắp xếp theo quy tắc riêng 60
Sắp theo nhiều tiêu chí 60
10.5. pair - struct hai trường có sẵn 61
10.6. Structured binding - mở gói cho gọn 61
10.7. auto - để máy tự đoán kiểu 61
10.8. Vài hàm có sẵn nên biết ngay 62
10.9. Bạn đã đi tới đâu - và đi tiếp thế nào 62
10.10. Bẫy của Phần 10 63
10.11. Bài tập 63
Phần 11. Học để thi đấu 64
11.1. Một bài thi trông như thế nào 64
11.2. Giới hạn n nói cho bạn biết đáp án 64
11.3. Các loại kết quả từ máy chấm 64
11.4. Danh sách kiểm tra trước khi nộp 65
11.5. Chiến thuật trong phòng thi 65
11.6. Luyện tập ở đâu 65
Phần 12. Tìm và sửa lỗi (debug) 67
12.1. Ba loại lỗi - nhớ lại và cách nhận diện 67
12.2. Lỗi biên dịch - đọc cho đúng 67
12.3. Kĩ thuật 1: in ra giữa chừng 68
12.4. Kĩ thuật 2: thu nhỏ dữ liệu 68
12.5. Kĩ thuật 3: stress test - vũ khí mạnh nhất 68
12.6. Kĩ thuật 4: bật sanitizer - máy tự chỉ chỗ sập 69
12.7. Kĩ thuật 5: assert - chốt chặn tự động 70
12.8. Kĩ thuật 6: đọc lại code cho vịt nghe 71
12.9. Quy trình debug - làm theo khi bí 71
12.10. Vài lỗi kinh điển và triệu chứng của chúng 71
12.11. Trường hợp biên hay quên 72
12.12. Ba câu về debug 72
Phần 13. Tra cứu 73
13.1. Sổ tay cú pháp - dán lên tường 73
13.2. Bảng 25 bẫy - đọc trước khi nộp bài 74
13.3. Lộ trình 30 ngày 75
13.4. Đi tiếp 75
13.5. Ba câu để lại 76

Lê Thành Nam

GIỚI THIỆU

Tài liệu này dành cho ai?

Cho bạn - người chưa từng viết một dòng code nào trong đời.

Không phải "đã học Pascal rồi chuyển sang C++". Không phải "đã biết chút Python". Chưa biết gì thật. Chưa từng mở một trình soạn thảo code. Chưa biết chương trình máy tính là cái gì. Đó chính xác là người mà tài liệu này được viết cho.

> **GHI NHỚ** **Mình sẽ không giả định bạn biết bất cứ điều gì.** Mỗi khái niệm mới đều được giải thích từ đầu, bằng ví dụ đời thường trước, rồi mới tới code. Nếu có chỗ nào bạn thấy "tự nhiên nhảy cóc" - đó là lỗi của mình, không phải của bạn.

Nhưng tài liệu này có một mục tiêu rõ ràng: đưa bạn tới chỗ THI ĐẤU được. Nghĩa là mình sẽ dạy C++ theo cách hơi khác các sách giáo khoa thông thường.

Tài liệu này KHÔNG dạy gì - và vì sao

C++ là một ngôn ngữ khổng lồ. Học hết nó mất nhiều năm. Nhưng để thi lập trình, bạn chỉ cần khoảng 20% của nó. Mình sẽ dạy đúng 20% đó, thật kĩ, và bỏ hẳn 8[illegible] còn lại.

Thứ bị bỏ	Vì sao bỏ
Lập trình hướng đối tượng (class, kế thừa, đa hình)	Gần như không dùng trong thi đấu. Học sau cũng được.
Con trỏ (pointer), cấp phát động	`vector` và tham chiếu thay thế hoàn toàn, an toàn hơn nhiều
Template tự viết, metaprogramming	Dùng template có sẵn là đủ
Xử lí ngoại lệ (try/catch)	Bài thi không cần bắt lỗi runtime
Đa luồng, mạng, giao diện	Không liên quan tới thi đấu
`char*` và mảng kí tự kiểu C	`string` dễ hơn và đủ nhanh

> **GHI NHỚ** **Đây không phải "học tắt".** Đây là chọn đúng thứ cần học trước. Một người viết được vòng lặp và mảng thật vững sẽ thi tốt hơn nhiều so với người biết tên 50 khái niệm mà không dùng thạo cái nào.

> **MẸO THI** **Nếu sau này bạn muốn làm phần mềm chuyên nghiệp,** hãy quay lại học con trỏ và hướng đối tượng. Chúng quan trọng - chỉ là không quan trọng *bây giờ*.

Vì sao là C++ mà không phải ngôn ngữ khác?

Câu hỏi công bằng. Python dễ học hơn nhiều. Java an toàn hơn. Vậy sao các kì thi tin học từ cấp tỉnh tới quốc tế đều dùng C++?

- **Tốc độ.** C++ chạy nhanh hơn Python khoảng 50–100 lần. Một bài mà C++ chạy 0,5 giây thì Python cần 30 giây - và đề chỉ cho 1 giây. Đây là lí do quyết định.
- **Thư viện chuẩn STL.** Sắp xếp, tập hợp, hàng đợi ưu tiên - có sẵn, viết một dòng, và nhanh.

Lê Thành Nam

- **Là ngôn ngữ chính thức của mọi kì thi.** Kì thi quốc tế IOI, kì thi quốc gia HSGQG, các trang luyện tập - tất cả đều lấy C++ làm chuẩn.

Cái giá phải trả: C++ khó hơn, và nó cho phép bạn tự bắn vào chân mình bằng nhiều cách. Nhưng tài liệu này sẽ chỉ cho bạn từng cái bẫy một, kèm bằng chứng cụ thể.

Bạn cần chuẩn bị gì

Rất ít:

- **Một trình biên dịch.** Trên Windows: cài Dev-C++ hoặc Code::Blocks (đã có sẵn g++ bên trong). Trên Linux: gõ `sudo apt install g++`. Trên máy Mac: cài Xcode Command Line Tools.
- **Một chỗ để gõ code.** Chính Dev-C++ hoặc Code::Blocks là đủ. Sau này có thể chuyển sang VS Code cho thoải mái hơn.
- **Giấy và bút.** Nghiêm túc đấy. Bạn sẽ cần vẽ ra để hiểu mảng và đệ quy.

> **GHI NHỚ** **Tài liệu này dùng g++ phiên bản 13 với chuẩn C++17.** Mọi kết quả in ra trong khung xám là kết quả máy thật chạy, mình không gõ tay. Nếu máy bạn cho kết quả khác ở những chỗ mình đánh dấu "giá trị rác" - đó là bình thường, mình sẽ giải thích khi tới đó.

Cách đọc tài liệu này

Mỗi phần đều theo mạch: khái niệm → ví dụ đời thường → code chạy được → kết quả thật → cái bẫy đi kèm.

Ba loại khung màu bạn sẽ gặp liên tục:

> **GHI NHỚ** Khung xanh - điều cần nhớ kĩ. Đây là ý chính, đọc chậm.

> **⚠ BẪY** Khung cam - cái bẫy. Đây là chỗ người mới hay sai. Mình sẽ chứng minh bằng code chạy thật.

> **MẸO THI** Khung xanh lá - mẹo thi đấu. Thứ giúp bạn viết nhanh hơn hoặc tránh mất điểm oan.

> **⚠ BẪY** **Quy tắc quan trọng nhất: GÕ LẠI MỌI ĐOẠN CODE.** Đừng chỉ đọc. Đừng copy. Hãy gõ tay từng chữ, chạy thử, rồi cố tình sửa sai một chỗ xem máy báo gì. Bạn sẽ học được từ 10 phút gõ tay nhiều hơn 2 tiếng đọc lướt. Lập trình là kĩ năng vận động, không phải kiến thức.

Lê Thành Nam